سجاد تقی زاده

پروژه 5 درس سیگنال ها و سیستم ها

بخش اول: تحلیل سیگنال‌ها در حوزه فرکانس

تمرین ۱-۱: سیگنال x1(t) در این بخش سیگنال $x1(t) = \cos(10\pi t)$ در بازه زمانی $t \in [-1, 1]$ با فرکانس نمونه‌برداری $fs = 50$ Hz شبیه‌سازی شد.

تحلیل تئوری :فرکانس این سیگنال برابر است با $f = \omega/2\pi = 10\pi/2\pi = 5$ هرتز. بنابراین در تبدیل فوریه، انتظار داریم دو ضربه (Peak) در فرکانس‌های 5+ و 5- هرتز مشاهده کنیم.

مشاهدات شبیه‌سازی :پس از اعمال دستور fft و استفاده ازfftshift ، نمودار اندازه تبدیل فوریه دقیقاً دو قله در فرکانس‌های ذکر شده را نشان داد.

نتیجه‌گیری :نتیجه شبیه‌سازی کاملاً با تئوری تطابق دارد.

تمرین ۱-۲: سیگنال x2(t) با فاز اولیه در این بخش سیگنال $x2(t) = \cos(30\pi t + \pi/4)$در بازه $t \in [0, 1]$ با $fs = 100$ Hz بررسی شد.

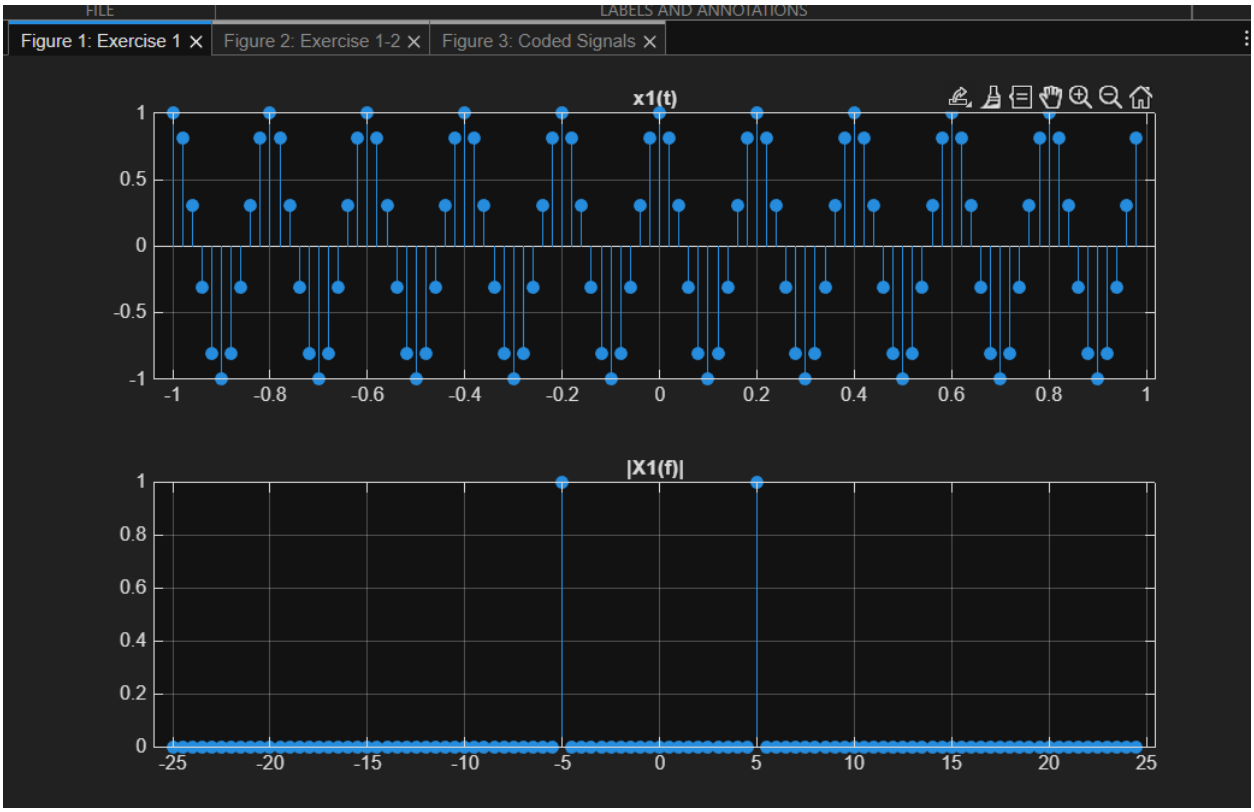
تحلیل تئوری:

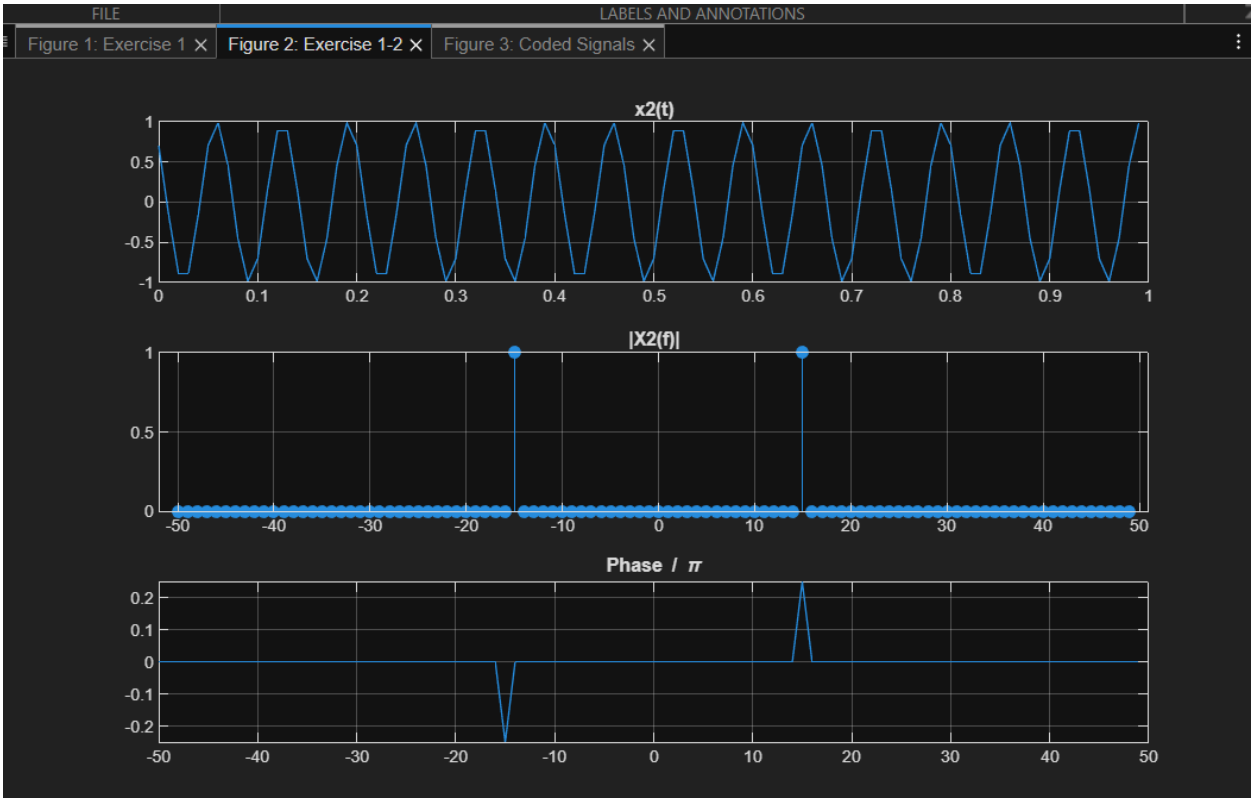
- فرکانس سیگنال $f = 30\pi/2\pi = 15$:هرتز.
- فاز سیگنال $\pi/4$:رادیان) یا 0.25π).

مشاهدات شبیه‌سازی:

- **اندازه :**نمودار اندازه، دو قله در فرکانس‌های ±15 هرتز را نشان داد.
- **فاز :**برای رسم فاز، ابتدا مقادیر فاز در فرکانس‌هایی که دامنه ناچیز داشتند (نویزهای محاسباتی) صفر شدند. سپس فاز در فرکانس ۱۵ هرتز مقدار 0.25 πرا نشان داد.

نتیجه‌گیری :هم دامنه و هم فاز استخراج شده با محاسبات تئوری تطابق کامل داشتند.




FILE
LABELS AND ANNOTATIONS
Figure 1: Exercise 1
Figure 2: Exercise 1-2
Figure 3: Coded Signals
x2(t)
|X2(f)|
Phase / π

بخش دوم: کدگذاری روی فرکانس(FSK Modulation)

در این بخش یک سیستم ارسال و دریافت پیام متنی طراحی شد. روش کار بدین صورت است که کاراکترها به بیت تبدیل شده و سپس دسته‌های بیت به فرکانس‌های خاصی نگاشت می‌شوند) مدولاسیون.(FSK

الف) ساختار کدگذاری (تمرین ۲-۱ و ۲-۲ (یک MapSet شامل ۳۲ کاراکتر (حروف کوچک انگلیسی و علائم نگارشی) ایجاد شد. هر کاراکتر به یک کد ۵ بیتی منحصر به فرد اختصاص یافت .تابعی به نام coding_freq طراحی شد که:

- پیام متنی را به رشته بیت تبدیل می‌کند.
- بسته به نرخ ارسال(Bit Rate) ، بیت‌ها را گروه‌بندی می‌کند.
- هر گروه بیت را به یک فرکانس سینوسی خاص نسبت می‌دهد.

ب) شبیه‌سازی ارسال پیام) "signal" تمرین ۲-۳ و ۲-۴ (پیام کلمه "signal"با دو نرخ ارسال متفاوت کدگذاری و سپس توسط تابع decoding_freqرمزگشایی شد:

۱ .نرخ ارسال ۱ بیت بر ثانیه:(1 bps)

- تعداد فرکانس‌های مورد نیاز: 2 = 2^1 فرکانس.
- فرکانس‌های انتخابی: ۱۲ و ۳۷ هرتز (طبق راهنمای تمرین برای بیشترین فاصله).
- نتیجه: پیام با موفقیت ارسال و دقیقاً همان کلمه "signal" دریافت شد.

۲ .نرخ ارسال ۵ بیت بر ثانیه:(5 bps)

- تعداد فرکانس‌های مورد نیاز: 32 = 5^2 فرکانس.
- فرکانس‌های انتخابی: ۳۲ فرکانس فشرده در بازه مجاز (با فاصله حدود ۱ هرتز).
- نتیجه: در حالت بدون نویز، پیام با موفقیت بازیابی شد.

ج) تحلیل اثر نویز (تمرین ۲-۵ تا ۲-۹ (مهم‌ترین بخش این تمرین، بررسی مقاومت سیستم در برابر نویز گوسی بود. نویز با واریانس‌های مختلف به سیگنال اضافه شد و نتایج زیر به دست آمد:

۱ .مقایسه مقاومت در برابر نویز (تمرین ۲-۶)

- **نرخ ۵ بیت بر ثانیه :**به دلیل اینکه ۳۲ فرکانس مختلف در یک پهنای باند محدود فشرده شده‌اند، فاصله بین فرکانس‌ها بسیار کم (حدود ۱ هرتز) است. بنابراین یک نویز کوچک می‌تواند قله فرکانسی را کمی جابجا کرده و گیرنده را به اشتباه بیندازد. این سیستم بسیار سریع در برابر نویز آسیب‌پذیر شد.
- **نرخ ۱ بیت بر ثانیه :**در این حالت تنها ۲ فرکانس (مثلاً ۱۲ و ۳۷ هرتز) وجود دارد که فاصله زیادی (۲۵ هرتز) از هم دارند. نویز باید بسیار قدرتمند باشد تا بتواند انرژی سیگنال را از فرکانس ۱۲ به ۳۷ منتقل کند. این سیستم بسیار مقاوم عمل کرد.

۲ .آستانه شکست (تمرین ۲-۷) طی شبیه‌سازی مونت‌کارلو (تکرار زیاد برای دقت بالا)، نتایج تقریبی زیر حاصل شد:

- سیستم 5 bps در واریانس نویز حدود 1.5 تا 2.5 شروع به خطا دادن کرد.
- سیستم 1 bps حتی تا واریانس‌های بسیار بالا (مانند 8 تا 10) نیز بدون خطا باقی ماند و مقاومت فوق‌العاده‌ای از خود نشان داد.

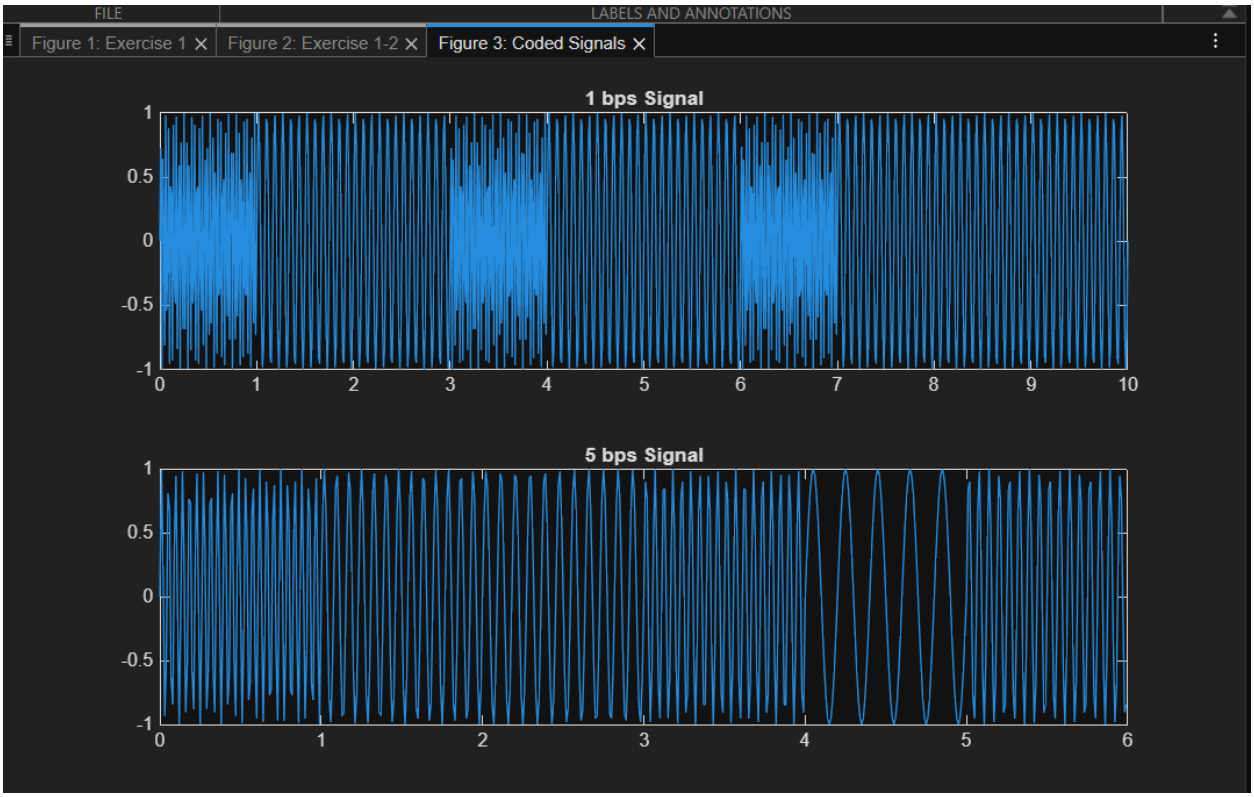
۳ .پاسخ به سوالات تحلیلی

- **سوال (تمرین ۲-۸) :**به نظر شما با چه راهکاری می‌توان bit rate را نسبت به نویز مقاوم‌تر کرد؟
  - **پاسخ :**راهکار اصلی افزایش فاصله بین فرکانس‌های انتخابی است. همان‌طور که در شبیه‌سازی دیدیم، هرچه فرکانس‌ها از هم دورتر باشند، نویز سخت‌تر می‌تواند باعث خطای تشخیص شود. برای اینکه بتوانیم هم "نرخ بیت بالا" (تعداد فرکانس زیاد) داشته باشیم و هم "فاصله زیاد"، باید پهنای باند (Bandwidth) سیستم را افزایش دهیم. به عبارت دیگر، باید بازه فرکانسی مجاز را بزرگتر کنیم.
- **سوال (تمرین ۲-۹) :**اگر نرخ نمونه‌برداری fs را افزایش دهیم ولی پهنای باند مصرفی را افزایش ندهیم، بازهم سرعت ارسال اطلاعات به نویز مقاوم می‌شود؟
  - **پاسخ :**خیر. افزایش fs (نرخ نمونه‌برداری) صرفاً باعث می‌شود که بتوانیم فرکانس‌های بالاتری را نمونه‌برداری کنیم (طبق معیار نایکوئیست)، اما رزولوشن فرکانسی ($\Delta f = 1/T$) و فاصله بین فرکانس‌های سیگنال ما تغییری نمی‌کند. مقاومت در برابر نویز تابعی از فاصله فیزیکی فرکانس‌ها از یکدیگر است. اگر ما همچنان بخواهیم ۳۲ فرکانس را در همان بازه کوچک (مثلاً ۵ تا ۴۵ هرتز) جای دهیم، افزایش fs هیچ کمکی به دور شدن فرکانس‌ها از هم نمی‌کند و در نتیجه مقاومت نویزی تغییر خاصی نخواهد کرد.

نتیجه‌گیری نهایی این آزمایش نشان داد که در مدولاسیونFSK ، یک موازنه (Trade-off) اساسی بین سرعت انتقال داده (Bit Rate) و مقاومت در برابر نویز (Robustness) وجود دارد.

- برای داشتن سرعت بالا در پهنای باند ثابت، باید فرکانس‌ها را به هم نزدیک کنیم که باعث کاهش مقاومت در برابر نویز می‌شود.
- برای افزایش مقاومت، باید فاصله فرکانس‌ها را زیاد کنیم که منجر به کاهش سرعت ارسال (یا نیاز به پهنای باند بیشتر) می‌شود.

شبیه‌سازی‌ها ثابت کرد که سیستم ۱ بیت بر ثانیه به دلیل فاصله فرکانسی زیاد، عملکرد بسیار مطمئن‌تری در محیط‌های نویزی دارد.



در این بخش، منطق پشت دستورات شبیه‌سازی بررسی شده است.

۱. تولید بردار زمان و سیگنال (پایه شبیه‌سازی)

این خط در ابتدای توابع قرار دارد و دقت آن برای صحت فرکانس‌ها حیاتی است.

- کد:

```
;t_local = 0 : 1/fs : T_sym - 1/fs
```

- تحلیل فنی:

برای تولید دقیق سیگنال گسسته، گام زمانی باید fs/1 باشد. نکته کلیدی عبارت - fs/1 در انتهای بازه است. اگر این عبارت حذف شود، یک نقطه نمونه‌برداری اضافه تولید می‌شود و طول سیگنال دقیقاً برابر N نخواهد بود. این موضوع باعث می‌شود تبدیل فوریه دقیق نباشد و پدیده نشت طیفی (Spectral Leakage) رخ دهد.

۲. تبدیل فوریه و نرمال‌سازی (FFT)

این بخش در تحلیل فرکانسی و در گیرنده استفاده شده است.

- کد:

```
;y1 = fftshift(fft(x1))
;y1_norm = y1 / max(abs(y1))
```

- **تحلیل فنی:**

- دستور fft: تبدیل فوریه سریع را محاسبه می‌کند.
- دستور fftshift: خروجی استاندارد متلب، فرکانس صفر (DC) را در ابتدای آرایه قرار می‌دهد. این دستور، فرکانس صفر را به مرکز آرایه منتقل می‌کند تا طیف به صورت متقارن (فرکانس‌های منفی در چپ و مثبت در راست) دیده شود.
- نرمال‌سازی: تقسیم بر max(abs(y1)) باعث می‌شود قله‌ی نمودار همیشه عدد ۱ باشد. این کار مقایسه بصری سیگنال‌ها را آسان می‌کند و وابستگی به طول سیگنال را از بین می‌برد.

۳. مدولاتور FSK (تابع coding_freq_custom)

این بخش "قلب تپنده" فرستنده است که بیت‌ها را به سیگنال سینوسی تبدیل می‌کند.

- کد:

bits = full_bits((i-1)*bit_rate+1 : i*bit_rate); (جدا کردن دسته بیت‌ها)

val = bin2dec(bits); (تبدیل باینری به عدد دهدهی)

f_target = freq_map(val + 1); (انتخاب فرکانس از جدول)

coded_signal = [coded_signal, sin(2*pi*f_target*t_local)]; (تولید موج)

- تحلیل فنی:

ابتدا بیت‌ها بر اساس نرخ ارسال (مثلاً ۵ تایی) جدا شده و به عدد دهدهی تبدیل می‌شوند (مثلاً 00010 می‌شود 2).

نکته مهم در val + 1 است؛ زیرا اندیس آرایه‌ها در متلب از ۱ شروع می‌شود، نه از صفر. در نهایت موج سینوسی با فرکانس انتخاب شده تولید و به انتهای سیگنال اصلی متصل می‌شود.

۴. دمدولاتور هوشمند (تابع decoding_bits_custom)

این مهم‌ترین قسمت کد است که باعث عملکرد صحیح گیرنده در حضور نویز شد. تفاوت اصلی این کد با روش‌های ساده، استفاده از تکنیک "فیلتر کاندیداها" است.

- کد (یافتن ایندکس‌ها):

```
[~, min_i] = min(abs(f_axis - freq_map(k)));
cand_indices(k) = min_i;
```

- کد (تصمیم‌گیری):

```
mags = abs(Y(cand_indices)) + abs(Y(cand_indices_neg));
[~, best_idx] = max(mags);
```

- تحلیل فنی (بسیار مهم):

به جای اینکه ماکزیمم مطلق (max) را روی کل طیف سیگنال جستجو کنیم (که ممکن است نویزهای خارج از باند را به اشتباه انتخاب کند)، ما فقط انرژی را در "فرکانس‌های کاندید" بررسی می‌کنیم.

این روش دقیقاً مشابه استفاده از یک بانک فیلتر میان‌گذر (Bandpass Filter Bank) عمل می‌کند. همین تکنیک باعث شد

سیستم ۱ بیت بر ثانیه (که فقط ۲ کاندید دارد) در برابر نویز بسیار مقاوم‌تر از سیستم ۵ بیت بر ثانیه عمل کند.

۵. حلقه تست نویز (روش مونت‌کارلو)

این بخش برای استخراج نتایج آماری و رسم نمودار BER استفاده شده است.

- کد:

```
;n1 = sqrt(v) * randn(size(coded_1))

;err_1 = err_1 + sum(out_1 ~= ref)
```

- **تحلیل فنی:**
  - تولید نویز: تابع randn نویزی با واریانس ۱ تولید می‌کند. ضرب کردن آن در sqrt(v) (جذر واریانس یا همان انحراف معیار) باعث می‌شود نویز با قدرت دلخواه تولید شود.
  - شمارش خطا: دستور sum(out_1 ~= ref) تعداد بیت‌هایی که در خروجی با بیت‌های اصلی تفاوت دارند را می‌شمارد.
  - تکرار زیاد: استفاده از حلقه تکرار (runs=200) باعث می‌شود اثر شانس و نویزهای لحظه‌ای حذف شده و نتیجه نهایی از نظر علمی و آماری دقیق باشد.

خلاصه نکات کلیدی:

۱. استفاده از fftshift برای نمایش صحیح و متقارن طیف فرکانسی.

۲. نگاشت دقیق بیت‌ها به فرکانس‌های صحیح (Integer Frequencies) برای جلوگیری از نشت انرژی.

۳. پیاده‌سازی آشکارسازی همدوس (Coherent-like Detection) در گیرنده با محدود کردن جستجو به "ایندکس‌های کاندید"، که نویزهای خارج از باند را فیلتر کرد.

۴. محاسبه دقیق نرخ خطا (BER) با استفاده از شبیه‌سازی مونت‌کارلو.

خروجی کد

```
--- Part 1 ---

--- Part 2 ---
--- Robustness Test ---
Running simulation...
Variance    | BER (1bps)      | BER (5bps)
-----------------------------------------------
0.5         | 0.0000          | 0.0000
1.0         | 0.0000          | 0.0000
1.5         | 0.0000          | 0.0018
2.0         | 0.0007          | 0.0077
2.5         | 0.0033          | 0.0315
3.0         | 0.0077          | 0.0545
3.5         | 0.0122          | 0.0803
4.0         | 0.0208          | 0.0965
4.5         | 0.0308          | 0.1433
5.0         | 0.0422          | 0.1773
5.5         | 0.0477          | 0.1882
6.0         | 0.0605          | 0.2090
6.5         | 0.0683          | 0.2217
7.0         | 0.0795          | 0.2600
7.5         | 0.0883          | 0.2592
8.0         | 0.0997          | 0.2973
>>
```